

Design Document

Distributed Prime Number Finder AFS File System

1. Introduction

The system finds prime numbers from very large data files. It is built with three main parts:

- A. RPC based File System: stores and replicates files safely.
- B. Coordinator: assigns work to workers and collects results.
- C. Workers: process file chunks and find prime numbers.

All parts talk to each other and run inside Docker containers.

2. Goals

Our goal is to create an app capable of handling large datasets in a safe and persistent manner.

For that we derived a general set of objectives:

- A. Build a fault-tolerant distributed file system using RPC protocols.
- B. Implement a distributed Coordinator–Worker model for prime search.
- C. Design recovery using snapshot-based global state saving.
- D. Test the system to meet the five success metrics: survive, recover, deliver, scale, and perform.

3. Assumptions

For the purposes of this project we assume that:

1. Input files are already on the server and its replicas at startup.
2. There is no disk corruption.
3. All workers have equal characteristics (memory, processing power etc).

4. Infrastructure Setup

1. Built using Python, gRPC, and Docker.
2. 3 file servers run as file service replicas.
3. File Service 1 becomes the default leader, others are followers.
4. A Coordinator container connects to FS leader.
5. Many Worker containers connect to both Coordinator and FS.
6. Each File Servers and Coordinator runs on static pre-defined ports.
7. Environment variables are used to link all services together.

5. Fault Tolerance in Infrastructure

1. Leader Election: If FS leader fails, followers hold a Raft-like election and pick a new leader. Each candidate increases its term, votes for itself, and asks peers for votes. Only becomes leader after securing majority. This prevents split-brain and coordinates leadership to serialize writes without needing Raft's full replicated log protocol.
2. Replication: When leader receives a write it attempts to copy the data to all followers, but it requires acknowledgements from a quorum (majority) before proceeding. Replication to every follower is attempted by reaching a majority.
3. Quorum-based commit (leader-driven two-step): The leader first writes the upload to a local temporary file, then replicates the temp file to followers. After receiving acknowledgements from a majority, it atomically renames (commits) its temp file to the final filename and updates metadata. Finally, the leader notifies followers (best-effort) to rename their tmp -> final. This guarantees that the file is either fully visible everywhere or nowhere (preventing corruption if leader crashes mid-write).
4. Retry Mechanism: Clients retry RPC calls with exponential backoff for server failures and have limited retries. If the server it contacts is not the leader, then this server includes a hint to discover it.

5. Atomic Writes: All nodes write incoming data to a temp file first, then perform an atomic rename (os.replace) to the final file on commit. This simplifies recovery by removing tmp files on startup.
6. What happens when a leader crashes: The leader stops sending heartbeats. Followers track the last heartbeat and, after a randomized election timeout, a follower becomes a candidate, bumps its term, votes for itself, and requests votes. If it wins a majority, it becomes leader, starts sending heartbeats, and future client writes should be redirected to this new leader.
7. Leader returns success to the client once the leader has locally committed and replicated to a majority.

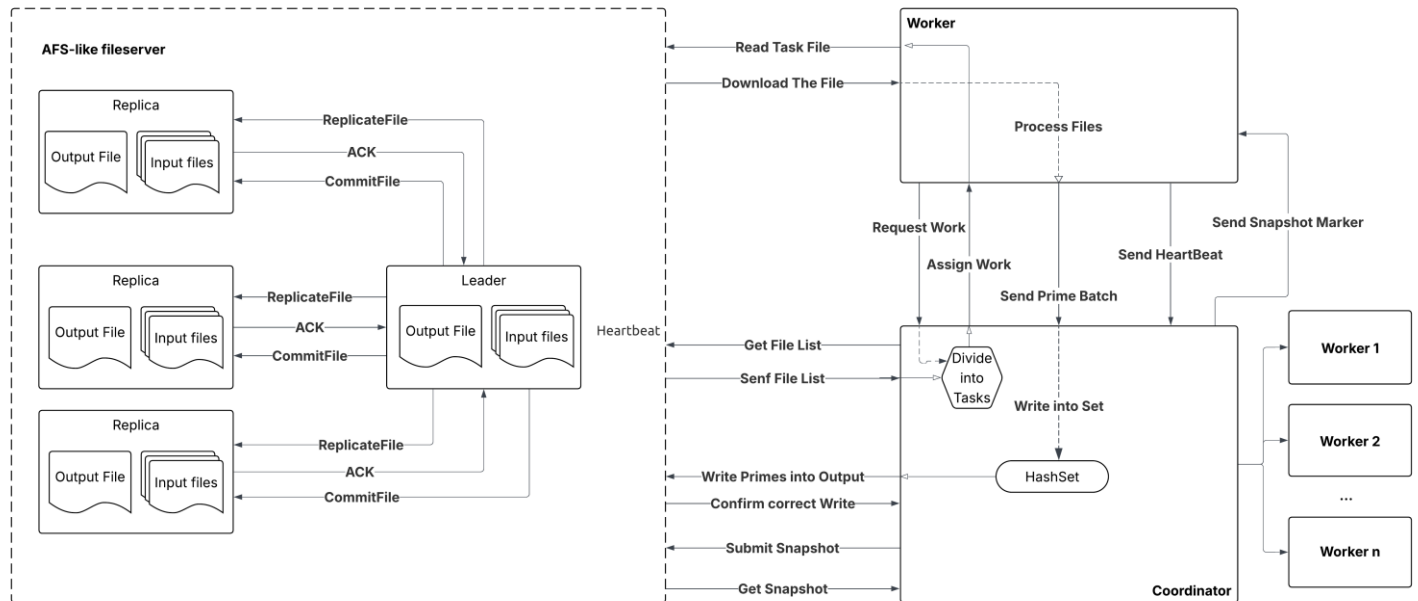


Chart meant as a generalised representation of our solution

6. Application Architecture

1. Coordinator–Worker Design: Coordinator divides the file into chunks and assigns it to the workers for processing.
 - A. Simplicity for dealing with concurrent writes to the primes.txt compared to a peer-to-peer solution.
 - B. However, the coordinator creates a single point of failure within the application.
2. Subdivisions: Very large input files are split into smaller input files such that the worker doesn't have to store can still store it in memory. This allows to limit the memory load on the worker at the potential expense of multiple downloads influencing performance.
3. Task Splitting: Input files are split into smaller chunks, increasing the frequency of output being uploaded, which limits potential losses on any application failure.
4. Task Queue: Coordinator keeps track of all tasks, the state of the task, and which worker has which task assigned.
5. Cache Aware scheduling: when assigning tasks the coordinator will give preference to those which the worker already has input file of.
6. Prime Collection: Workers send back primes output and then the coordinator stores them in one file.
7. Uniqueness Check: Coordinator keeps a set of all primes found to avoid duplicates.
8. Global Snapshot: Coordinator will initiate a snapshot every 30 seconds using the Chandy Lamport algorithm.
 - A. It will first record its own state containing the set of primes, current tasks in progress, and other data structures related to the cache aware scheduling.
 - B. It sends markers to all active workers and starts recording in-flight messages on all channels.
 - C. When the worker receives a marker for a new snapshot, it records its own state consisting of the current task it is working on (task id, corresponding input file, start and end lines) and sends it to the coordinator.
 - D. Once the coordinator has received all worker states, it persists the global snapshot to the fileserver.
9. When a coordinator initially starts, it will first check if there is a snapshot on the fileserver, and if there is it will recover from that snapshot.

7. Data Flow

1. The coordinator starts and connects to FileService leader.
2. The coordinator fetches all metadata of the input files and splits them into chunks according to their size.
3. The workers connect to the coordinator and request work.
4. The coordinator sends the task id, filename, start line, and number of lines to a worker.
5. The workers download that file from the File Service Leader using OpenFile RPC function.
6. The worker checks numbers in that range and finds primes.
7. The worker sends the prime list back to Coordinator which handles deduplication.
8. The coordinator requests that the File Service append the output in primes.txt file.
9. Snapshots of progress are saved every 30 seconds.

8. Data Fault Tolerance and Recovery

1. Whole File Caching: Workers store files locally and check timestamps using TestAuth RPC.
2. Snapshot Saving: Coordinator and workers save state regularly to File Service Leader.
3. Snapshot Loading: If a system crashes, it loads the latest snapshot from the file service leader and continues.
4. Majority Replication: File data is safe because it exists on replica servers.
5. Follower Sync: Followers updated from leader after recovery.
6. Automatic Redirect: Clients find the newly elected File Service leader using redirect messages.
7. Task Reassignment: If a worker fails, Coordinator gives that task to another worker.

9. Objectives Achieved

1. The system can survive worker, coordinator and file service failures.
2. It recovers full state from snapshots.
3. It keeps file data safe with multiple replications.
4. It scales easily by adding more workers.
5. It performs caching and retries.

6. It delivers final results (primes.txt) even after crashes.
7. It passes all five success metrics: Survive, Recover, Deliver, Scale, Perform.